

# BIZEV-99: Best Practice Summary

Series: BIZEV — Business Events & Funnel Analysis | Notebook: 99 | Created: March 2026 | Last Updated: 10/02/2026

## Overview

This notebook consolidates every actionable best practice from the BIZEV series (notebooks 01 through 07) into a single, definitive reference. Each practice specifies exactly what to set, with no hedging. Use this as a checklist when instrumenting business events, building conversion funnels, and creating executive reports in Dynatrace.

## Table of Contents

- 1. [Data Model and Schema](#)
- 2. [Ingestion and Instrumentation](#)
- 3. [Event Naming and Payload Design](#)
- 4. [Funnel Analysis](#)
- 5. [Revenue and Impact Analysis](#)
- 6. [KPIs and Metric Extraction](#)
- 7. [Executive Reporting and Dashboards](#)
- 8. [DQL Query Patterns](#)
- 9. [SLA and SLO Tracking](#)
- 10. [Gen2 vs Gen3 Adoption](#)

## Prerequisites

| Requirement           | Details                                                                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Dynatrace Environment | SaaS with Grail enabled                                                                                                                    |
| Permissions           | <code>storage:bizevents:read</code> , <code>storage:events:read</code> , <code>storage:metrics:read</code> , <code>bizevents.ingest</code> |
| Knowledge             | BIZEV-01 through BIZEV-07                                                                                                                  |

## 1. Data Model and Schema

| # | Best Practice                                                     | Recommended Setting/Value                                                                                   | Priority    | Category    |
|---|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------|-------------|
| 1 | Use <code>bizevents</code> for business transactions              | <code>fetch bizevents</code> — never use <code>events</code> or <code>logs</code> for business analytics    | Critical    | Data Source |
| 2 | Always populate <code>event.type</code>                           | Set to a reverse-domain string (e.g., <code>com.myapp.order.completed</code> ) on every event               | Critical    | Schema      |
| 3 | Always populate <code>event.provider</code>                       | Set to the source application or system name                                                                | Critical    | Schema      |
| 4 | Populate <code>event.category</code> on every event               | Set to a logical grouping value (e.g., <code>ecommerce</code> , <code>auth</code> , <code>payments</code> ) | Recommended | Schema      |
| 5 | Include a <code>user_id</code> or <code>session_id</code> field   | Required for funnel analysis and distinct-user conversion rates                                             | Critical    | Schema      |
| 6 | Include a numeric <code>amount</code> field on transaction events | Store as a number, never as a string — enables <code>sum()</code> , <code>avg()</code> without casting      | Critical    | Schema      |
| 7 | Use consistent field types across all events                      | Always send <code>amount</code> as double, <code>quantity</code> as long — never vary types                 | Recommended | Schema      |
| 8 | Never confuse <code>bizevents</code> with <code>events</code>     | <code>bizevents</code> = business transactions;<br><code>events</code> = infrastructure/operational signals | Critical    | Data Source |

## 2. Ingestion and Instrumentation

| #  | Best Practice                                 | Recommended Setting/Value                                                                                                                                  | Priority    | Category  |
|----|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------|
| 9  | Start with OneAgent auto-capture for web apps | Configure capture rules in <b>Settings &gt; Collect and Capture &gt; Business events</b>                                                                   | Critical    | Ingestion |
| 10 | Use CloudEvents format for API ingestion      | Content-Type: <code>application/cloudevents+json</code> with <code>specversion</code> , <code>type</code> , <code>source</code> , <code>data</code> fields | Critical    | Ingestion |
| 11 | Use batch ingestion for high-volume scenarios | Content-Type: <code>application/cloudevents-batch+json</code> — send arrays of events in a single POST                                                     | Recommended | Ingestion |

| #  | Best Practice                                                | Recommended Setting/Value                                                                                                                                                                                              | Priority    | Category   |
|----|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|------------|
| 12 | Implement retry logic with exponential backoff               | Retry on 429 and 5xx ; keep each request under the documented 5 MB payload limit (no request-rate figure is published)                                                                                                 | Recommended | Ingestion  |
| 13 | Use the RUM and mobile APIs for client-side capture          | <code>dynatrace.sendBizEvent(type, fields)</code> (RUM JavaScript API) or the mobile agents' equivalent; backend code without a capture rule uses the ingest API — there is no OneAgent SDK method for business events | Recommended | Ingestion  |
| 14 | Use OpenPipeline for span-to-bizevent mapping                | A <b>Business event</b> processor in the OpenPipeline <b>Data extraction</b> stage emits a new bizevent per matching span — no second instrumentation                                                                  | Recommended | Ingestion  |
| 15 | Never double-instrument                                      | If spans already carry business data, extract it with a Business event processor (Data extraction stage) — do not add a second telemetry call in code                                                                  | Critical    | Ingestion  |
| 16 | Verify ingestion health with a 15-minute interval timeseries | Run <code>makeTimeseries event_count = count(), interval:15m</code> daily to detect ingestion gaps                                                                                                                     | Recommended | Monitoring |
| 17 | Validate required field completeness after instrumentation   | Query <code>countIf(isNotNull(event.type)) / total</code> to confirm >99% field population                                                                                                                             | Critical    | Monitoring |

### 3. Event Naming and Payload Design

| #  | Best Practice                                    | Recommended Setting/Value                                                                                                              | Priority | Category |
|----|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------|----------|
| 18 | Use reverse-domain notation with action verb     | Format: <code>com.&lt;company&gt;.&lt;domain&gt;.&lt;action&gt;</code> (e.g., <code>com.myapp.order.created</code> )                   | Critical | Naming   |
| 19 | Use all-lowercase, dot-separated names           | <code>com.myapp.payment.processed</code> — never <code>PaymentProcessed</code> or <code>PAYMENT_PROCESSED</code>                       | Critical | Naming   |
| 20 | Never put version numbers in event type names    | Use payload fields for versioning — <code>com.myapp.order.created</code> stays the same, add <code>schema_version: 2</code> in payload | Critical | Naming   |
| 21 | Never use meaningless identifiers as event types | <code>event_12345</code> is forbidden — use descriptive domain names                                                                   | Critical | Naming   |

| #  | Best Practice                                                          | Recommended Setting/Value                                                                                               | Priority    | Category    |
|----|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------|-------------|
| 22 | Include business identifiers in payload                                | <code>order_id</code> , <code>customer_id</code> , <code>session_id</code>                                              | Critical    | Payload     |
| 23 | Include measurable values in payload                                   | <code>amount</code> , <code>quantity</code> , <code>duration_ms</code>                                                  | Critical    | Payload     |
| 24 | Include categorical dimensions in payload                              | <code>category</code> , <code>region</code> , <code>customer_tier</code>                                                | Recommended | Payload     |
| 25 | Never include high-cardinality strings in payload                      | No full URLs, stack traces, or free-text fields — parse into structured fields                                          | Critical    | Cardinality |
| 26 | Use high-cardinality fields for filtering only, not grouping           | <code>filter order_id == "ORD-123"</code> is fine;<br><code>summarize by:{order_id}</code> on millions of values is not | Critical    | Cardinality |
| 27 | Keep <code>summarize by:</code> dimensions under 1,000 distinct values | Fields like <code>customer_tier</code> (~3 values) or <code>product_category</code> (~100 values) are ideal             | Recommended | Cardinality |

## 4. Funnel Analysis

| #  | Best Practice                                                     | Recommended Setting/Value                                                                                                                                                                                            | Priority | Category      |
|----|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------------|
| 28 | Define explicit funnel steps as distinct event types              | Each step gets its own <code>event.type</code> :<br><code>product.viewed</code> → <code>cart.updated</code> →<br><code>checkout.started</code> →<br><code>payment.processed</code> →<br><code>order.completed</code> | Critical | Funnel Design |
| 29 | Use <code>countIf</code> for funnel step counts in a single query | <code>summarize {step1 = countIf(event.type == "..."), step2 = countIf(...)}</code> — never run separate queries per step                                                                                            | Critical | DQL Pattern   |
| 30 | Use <code>countDistinctApprox</code> for distinct-user funnels    | <code>countDistinctApprox(if(event.type == "...", then: user_id))</code> — more accurate than event counts for conversion rates                                                                                      | Critical | Funnel Design |
| 31 | Calculate step conversion as <code>step_N / step_N-1 * 100</code> | <code>round(toDouble(step2) / toDouble(step1) * 100, decimals: 1)</code>                                                                                                                                             | Critical | Metrics       |

| #  | Best Practice                                                                  | Recommended Setting/Value                                                                                                                                                                                  | Priority    | Category     |
|----|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|--------------|
| 32 | Calculate overall conversion as<br><code>last_step / first_step * 100</code>   | <code>round(toDouble(step5) / toDouble(step1) * 100, decimals: 2)</code>                                                                                                                                   | Critical    | Metrics      |
| 33 | Identify checkout abandonment with per-user aggregation                        | <code>summarize by:{user_id} then filter started_checkout &gt; 0 AND completed_payment == 0</code>                                                                                                         | Recommended | Analysis     |
| 34 | Measure time between funnel steps                                              | Convert timestamps to millis, subtract, divide by 1000 for seconds:<br><code>(unixMillisFromTimestamp(step2_time) - unixMillisFromTimestamp(step1_time)) / 1000.0</code>                                   | Recommended | Analysis     |
| 35 | Report avg , median , and p95 for inter-step timing                            | All three are needed to understand the distribution of user behavior                                                                                                                                       | Recommended | Analysis     |
| 36 | Segment funnels by <code>event.provider</code> and <code>event.category</code> | Reveals which channels and business lines convert best                                                                                                                                                     | Recommended | Segmentation |
| 37 | Track daily conversion rate trends over 7+ days                                | Use <code>bin(timestamp, 24h)</code> with <code>summarize by:{day}</code> to measure campaign/release impact ( <code>1d</code> is rewritten to <code>24h</code> with a notification; days are UTC-aligned) | Recommended | Trending     |

## 5. Revenue and Impact Analysis

| #  | Best Practice                                             | Recommended Setting/Value                                                                                           | Priority    | Category        |
|----|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------|-----------------|
| 38 | Correlate detected problems with business event volume    | Overlay <code>fetch dt.davis.problems</code> timelines with <code>fetch bizevents</code> volume to visualize impact | Critical    | Incident Impact |
| 39 | Use <code>spread:</code> for concurrent problem timelines | <code>makeTimeseries count = count(), spread: timeframe(from: event.start, to: coalesce(event.end, now()))</code>   | Recommended | DQL Pattern     |
| 40 | Compare incident periods against baselines                | Compare same hour yesterday or same day last week using <code>bin(now(), 24h)</code> offsets                        | Critical    | Incident Impact |

| #  | Best Practice                                                                                   | Recommended Setting/Value                                                                                                                                                                                                                                 | Priority    | Category        |
|----|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------------|
| 41 | Filter to business hours for impact assessment                                                  | Pass your timezone, or 9–5 is UTC:<br><code>getDayOfWeek(timestamp, timezone: "&lt;tz&gt;") &lt;= 5 AND<br/> getHour(timestamp, timezone: "&lt;tz&gt;") &gt;= 9 AND<br/> getHour(timestamp, timezone: "&lt;tz&gt;") &lt; 17</code> (ISO days: 1 = Monday) | Recommended | Context         |
| 42 | Convert Dynatrace Intelligence <code>resolved_problem_duration</code> from nanoseconds to hours | <code>resolved_problem_duration / 1h</code><br>— the field is in nanoseconds, not milliseconds                                                                                                                                                            | Critical    | Data Conversion |
| 43 | Exclude duplicate problems from impact analysis                                                 | <code>filter dt.davis.is_duplicate == false</code> . Do not add <code>dt.davis.is_frequent_event</code> — frequent issue detection is being phased out and the flag no longer separates anything (ADOPT-03 §5)                                            | Critical    | Data Quality    |
| 44 | Exclude maintenance windows from SLA calculations                                               | <code>filter maintenance.is_under_maintenance == false</code>                                                                                                                                                                                             | Recommended | Data Quality    |
| 45 | Use <code>sum(toDouble(amount))</code> for revenue calculations                                 | Always cast <code>amount</code> to double before aggregation                                                                                                                                                                                              | Critical    | DQL Pattern     |
| 46 | Track hourly revenue to detect incident-correlated dips                                         | <code>makeTimeseries hourly_revenue = sum(toDouble(amount)), interval:1h</code>                                                                                                                                                                           | Recommended | Monitoring      |

## 6. KPIs and Metric Extraction

| #  | Best Practice                                                         | Recommended Setting/Value                                                                                                                                         | Priority | Category   |
|----|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------|
| 47 | Define KPIs that are Specific, Measurable, Actionable, and Time-bound | Every KPI must answer a concrete business question with a formula                                                                                                 | Critical | KPI Design |
| 48 | Track these five core KPIs at minimum                                 | Transaction Volume ( <code>count()</code> ), AOV ( <code>avg(amount)</code> ), Conversion Rate, Error Rate, Revenue per Hour ( <code>sum(amount)</code> per hour) | Critical | KPI Design |
| 49 | Distinguish business error rates from HTTP error rates                | A payment can fail (business error) with HTTP 200 — use <code>event.type</code> to identify business failures, not HTTP status                                    | Critical | KPI Design |

| #  | Best Practice                                               | Recommended Setting/Value                                                                                                       | Priority    | Category          |
|----|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------|-------------------|
| 50 | Require minimum sample size before calculating error rates  | <code>filter total &gt; 10</code> before computing error percentages to avoid misleading rates                                  | Recommended | Data Quality      |
| 51 | Extract critical KPIs as Dynatrace metrics via OpenPipeline | Counter metric / Value metric processors in the OpenPipeline <b>Metric extraction</b> stage for order counts and revenue values | Critical    | Metric Extraction |
| 52 | Set metric dimensions to low-cardinality fields only        | Dimensions: <code>event.provider</code> , <code>product_category</code> — never <code>order_id</code> or <code>user_id</code>   | Critical    | Metric Extraction |
| 53 | Use extracted metrics for alerting and SLOs                 | Extracted metrics enable native Dynatrace Intelligence alerting and SLO definitions — DQL on <code>bizevents</code> does not    | Critical    | Alerting          |
| 54 | Build a composite business health score                     | Weight: Transaction Volume 30%, Error Rate 30%, AOV 20%, Conversion Rate 20% — thresholds: Green/Yellow/Red                     | Recommended | Health Scoring    |
| 55 | Health score thresholds: Error Rate                         | Green: < 1%, Yellow: 1–5%, Red: > 5%                                                                                            | Recommended | Health Scoring    |
| 56 | Health score thresholds: Volume vs Baseline                 | Green: > 90%, Yellow: 70–90%, Red: < 70%                                                                                        | Recommended | Health Scoring    |

## 7. Executive Reporting and Dashboards

| #  | Best Practice                                           | Recommended Setting/Value                                                                                  | Priority    | Category         |
|----|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------|------------------|
| 57 | Lead with business outcomes, not infrastructure metrics | Report transactions, revenue, conversion rates — never CPU or memory to executives                         | Critical    | Presentation     |
| 58 | Always provide baseline context                         | Compare against yesterday, last week, or last month — never show a number without comparison               | Critical    | Presentation     |
| 59 | Quantify incidents in business terms                    | "Lost 1,200 transactions and \$47,000 revenue" not "Service had 500ms latency for 2 hours"                 | Critical    | Presentation     |
| 60 | One KPI per dashboard tile                              | Each tile answers exactly one question — no multi-metric tiles                                             | Recommended | Dashboard Design |
| 61 | Include a data freshness indicator                      | <code>if(latest_event &gt; now() - 5m, then: "CURRENT", else: "STALE")</code> on every executive dashboard | Recommended | Dashboard Design |
| 62 | Executive dashboard must answer three questions         | 1) Is the business running? 2) Are there problems? 3) What is the trend?                                   | Critical    | Dashboard Design |

| #  | Best Practice                                                                                  | Recommended Setting/Value                                                                        | Priority    | Category         |
|----|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------|------------------|
| 63 | Automate report delivery via Workflows                                                         | Schedule daily reports at 8 AM weekdays with a Time trigger: <code>0 8 * * 1-5</code>            | Critical    | Automation       |
| 64 | Use <code>bin(now(), 24h) - 24h</code> to <code>bin(now(), 24h)</code> for "yesterday" queries | Produces clean 24-hour boundaries for scheduled reports                                          | Recommended | DQL Pattern      |
| 65 | Include a weekly business review query set                                                     | Daily volume trend, problems resolved with MTTR, week-over-week volume comparison, busiest hours | Recommended | Reporting        |
| 66 | Dashboard tile for current transaction rate                                                    | <code>count() / 15.0 over from:-15m =</code> transactions per minute                             | Recommended | Dashboard Design |
| 67 | Limit top-N lists to 10 items max in executive views                                           | <code>limit 10</code> on all ranked queries for readability                                      | Optional    | Dashboard Design |

## 8. DQL Query Patterns

| #   | Best Practice                                                            | Recommended Setting/Value                                                                                                                                                                                                          | Priority    | Category    |
|-----|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-------------|
| 68  | Always specify a time range on <code>fetch bizevents</code>              | <code>fetch bizevents, from:-24h</code> — never omit the time range                                                                                                                                                                | Critical    | Performance |
| 69  | Filter early, sort late, limit last                                      | Place <code>filter</code> immediately after <code>fetch</code> , <code>sort</code> after <code>summarize</code> , <code>limit</code> at the end                                                                                    | Critical    | Performance |
| 70  | Use <code>==</code> for exact matches, <code>~</code> only for wildcards | <code>filter event.type == "com.myapp.order.completed"</code> not <code>event.type ~ "*order*"</code>                                                                                                                              | Critical    | Performance |
| 71  | Use <code>in()</code> with curly-brace arrays for multi-value filters    | <code>filter in(event.type, {"type1", "type2"})</code> — never SQL-style <code>IN ('a', 'b')</code>                                                                                                                                | Critical    | Syntax      |
| 72  | Alias all aggregations used in sort                                      | <code>summarize c = count() then sort c desc</code> — never <code>sort count() desc</code>                                                                                                                                         | Critical    | Syntax      |
| 72b | Wrap multiple <code>summarize</code> aggregations in <code>{}</code>     | When summarizing 2+ aggregations, wrap them: <code>summarize { a = count(), b = avg(x) }, by: {field}</code> . Tenant emits <code>PARAMETERS_SHOULD_BE_GROUPED</code> INFO if not wrapped (query still runs but is non-canonical). | Recommended | Syntax      |

| #  | Best Practice                                                                           | Recommended Setting/Value                                                       | Priority    | Category     |
|----|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------------|--------------|
| 73 | Use <code>round(..., decimals: N)</code> with named parameter                           | <code>round(value, decimals: 2)</code> — never <code>round(value, 2)</code>     | Critical    | Syntax       |
| 74 | Use <code>if(cond, then: ..., else: ...)</code> with named parameters                   | Named <code>then:</code> and <code>else:</code> parameters are mandatory        | Critical    | Syntax       |
| 75 | Cast <code>amount</code> fields with <code>toDouble()</code> before math                | <code>sum(toDouble(amount))</code> — payload fields may arrive as strings       | Recommended | Data Types   |
| 76 | Use <code>isNotNull()</code> before aggregating optional fields                         | <code>filter isNotNull(amount)</code> before <code>sum(toDouble(amount))</code> | Critical    | Data Quality |
| 77 | Use <code>countDistinctApprox</code> over <code>countDistinct</code> for large datasets | Faster with negligible accuracy loss for >10,000 distinct values                | Recommended | Performance  |
| 78 | Target specific Grail buckets when possible                                             | <code>fetch bizevents, bucket: {"my_bizevents_*"}</code> to reduce scan volume  | Optional    | Performance  |

## 9. SLA and SLO Tracking

| #  | Best Practice                            | Recommended Setting/Value                                                                               | Priority    | Category |
|----|------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------|----------|
| 79 | Define a Transaction Success Rate SLO    | Target: 99.5% — Formula: <code>successful / total * 100</code> from payment events                      | Critical    | SLO      |
| 80 | Define a Business Availability SLO       | Target: 99.9% — Formula: hours with at least 1 business event / total hours * 100                       | Critical    | SLO      |
| 81 | Define an MTTR SLO                       | Target: < 2 hours — Formula: <code>avg(resolved_problem_duration)</code> in hours, excluding duplicates | Recommended | SLO      |
| 82 | Define a Revenue Impact per Incident SLO | Target: < \$5,000 — Formula: baseline revenue minus incident-period revenue                             | Recommended | SLO      |

| #  | Best Practice                                                                 | Recommended Setting/Value                                                                                                                                                                                   | Priority    | Category  |
|----|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------|
| 83 | Do not derive SLA availability from summed problem durations                  | Problems overlap, so summed durations are entity-hours of impact, not downtime — they can exceed the period many times over (BIZEV-04 §6). Define availability as an SLO on good ÷ total events (SLO-02/03) | Critical    | SLA       |
| 84 | Include SLO status indicator in every report                                  | <code>if(success_rate &gt;= 99.5, then: "MET", else: "MISSED")</code>                                                                                                                                       | Critical    | Reporting |
| 85 | Track MTTR with <code>avg</code> , <code>median</code> , and <code>p95</code> | Report all three — avg shows the mean, median shows the typical case, p95 shows worst cases                                                                                                                 | Recommended | Metrics   |

## 10. Gen2 vs Gen3 Adoption

Practices from BIZEV-07 for environments that have not (fully) moved to the Gen3 experience.

| #  | Best Practice                                                                                          | Recommended Setting/Value                                                                                                                                                                                                                                                                           | Priority    | Category |
|----|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 86 | Confirm the platform generation before designing business analytics                                    | Dynatrace Managed = no business events in any form (classic toolchain only); SaaS with Grail = adopt business events, regardless of how classic the day-to-day operation is                                                                                                                         | Critical    | Adoption |
| 87 | On Grail-enabled SaaS tenants, build new business analytics on business events, not classic mechanisms | Calculated service metrics are closed to new customers and superseded by OpenPipeline metric extraction — new classic investment is migration debt                                                                                                                                                  | Critical    | Adoption |
| 88 | Use the minimal Gen3 footprint for hybrid adoption                                                     | Capture via Settings (OneAgent rules — Full-Stack mode required), report in Notebooks + the new Dashboards app, grant <code>storage:bizevents:read</code> — nothing else has to move                                                                                                                | Recommended | Adoption |
| 89 | Bridge business KPIs to classic dashboards with metric extraction                                      | <code>bizevents.</code> -prefixed extracted metrics are readable in Data Explorer, classic dashboards, and classic metric-event alerting (classic pipeline: not available to accounts created from September 2026 and to be deactivated in a future release — build new extraction in OpenPipeline) | Recommended | Adoption |

| #  | Best Practice                                                         | Recommended Setting/Value                                                                                                                                                                                                                                                                         | Priority    | Category |
|----|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|
| 90 | Create metric-extraction rules at capture time, not reporting time    | Extraction is forward-only — no backfill; with classic-pipeline extraction, late-timestamped events land in a <code>.failed</code> -suffixed key                                                                                                                                                  | Critical    | Adoption |
| 91 | Capture must-not-miss events server-side, not (only) via RUM          | <code>sendBizEvent</code> is dropped for unmonitored sessions (cost and traffic controls) — orders and payments belong on OneAgent capture rules or the ingest API                                                                                                                                | Critical    | Adoption |
| 92 | On Managed, apply reverse-domain naming to classic artifacts          | Name request attributes, session properties, and log metrics in <code>com.&lt;company&gt;.&lt;domain&gt;.&lt;action&gt;</code> style so the eventual migration is a mapping exercise                                                                                                              | Recommended | Adoption |
| 93 | Size business events against your actual license model before rollout | Classic DDU: 100 DDUs/GB ingest, 0.30 DDUs/GB-day retain, 1.70 DDUs/GB query, 200,000 DDUs/year free tier; DPS: Events powered by Grail capability                                                                                                                                                | Recommended | Adoption |
| 94 | Scope business-analyst access with a bucket-limited IAM policy        | <code>ALLOW storage:bizevents:read WHERE storage:bucket-name = "default_bizevents";</code> plus the matching <code>storage:buckets:read</code> statement — grants Gen3-app access to business events only, nothing else in Grail; route sensitive streams to custom buckets for per-team policies | Recommended | Adoption |

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*